

Manual Técnico — AfterReload

Guía de instalación, configuración y administración del sistema

****Versión:**** 1.0

****Plataforma:**** Laravel 12 / PHP 8.2+

****Fecha:**** Mayo 2026

****Audiencia:**** Administradores de sistemas y desarrolladores

Índice

1. [Visión general del sistema](#1-visión-general-del-sistema)
2. [Requisitos del sistema](#2-requisitos-del-sistema)
3. [Instalación en Linux](#3-instalación-en-linux)
4. [Instalación en Windows](#4-instalación-en-windows)
5. [Configuración del entorno (.env)](#5-configuración-del-entorno-env)
6. [Configuración de la base de datos](#6-configuración-de-la-base-de-datos)
7. [Configuración de Nginx](#7-configuración-de-nginx)
8. [Configuración de PHP-FPM](#8-configuración-de-php-fpm)
9. [Integración con Steam OAuth](#9-integración-con-steam-oauth)
10. [Integración con Get5](#10-integración-con-get5)
11. [Integración RCON](#11-integración-rcon)
12. [Integración con servidor CS:GO local (LGSM)](#12-integración-con-servidor-csgo-local-lgsm)
13. [Compilación del frontend](#13-compilación-del-frontend)
14. [Comandos de mantenimiento](#14-comandos-de-mantenimiento)
15. [Gestión de logs](#15-gestión-de-logs)
16. [Gestión de la cola de trabajos](#16-gestión-de-la-cola-de-trabajos)
17. [Seguridad](#17-seguridad)
18. [Resolución de problemas](#18-resolución-de-problemas)
19. [Referencia de rutas del sistema](#19-referencia-de-rutas-del-sistema)

1. Visión general del sistema

AfterReload es una aplicación web desarrollada con **Laravel 12** orientada a la gestión de comunidades de Counter-Strike. Centraliza la autenticación de jugadores mediante Steam, la organización de lobbies para partidas matchmaking, el control remoto de servidores mediante RCON, el procesamiento automático de resultados y la administración de usuarios.

1.1 Componentes principales

Componente	Fichero	Responsabilidad
------------	---------	-----------------

#####

Autenticación Steam	`app/Http/Controllers/Auth/SteamController.php`	OAuth 2.0 con Steam
---------------------	---	---------------------

Gestión de lobbies	`app/Http/Controllers/Lobby/LobbyController.php`	Presencia, equipos, readiness
--------------------	--	-------------------------------

Webhook Get5	`app/Http/Controllers/Api/Get5Controller.php`	Eventos y resultados de match
--------------	---	-------------------------------

Panel admin	`app/Http/Controllers/Admin/AdminController.php`	CRUD de usuarios y servidores
-------------	--	-------------------------------

Cliente RCON	`app/Services/RconClient.php`	Protocolo RCON sobre TCP
--------------	-------------------------------	--------------------------

Procesador logs	`app/Services/PublicMatchResultProcessor.php`	Resultados en servidores públicos
-----------------	---	-----------------------------------

Runtime LGSM	`app/Services/LocalCsgoserverRuntimeService.php`	Estado de servidores locales
--------------	--	------------------------------

2. Requisitos del sistema

2.1 Requisitos mínimos (producción)

| Componente | Versión mínima | Notas |

#####

| PHP | 8.2 | Con extensiones indicadas |

| Composer | 2.x | Gestor de dependencias PHP |

| Node.js | 18 LTS o superior | Para compilar el frontend |

| npm | 9 o superior | Incluido con Node.js |

| Nginx | 1.18 o superior | Servidor web |

| SQLite | 3.x | Para instalaciones ligeras |

| MySQL / MariaDB | 8.0 / 10.6 | Recomendado para producción |

| Sistema operativo | Ubuntu 22.04+ / Debian 12+ | Linux recomendado |

| RAM | 512 MB mínimo | 1 GB recomendado |

| Disco | 500 MB libres | Sin contar el servidor de juego |

2.2 Extensiones PHP requeridas

php8.2-cli

php8.2-fpm

php8.2-mbstring

php8.2-xml

php8.2-curl

php8.2-sqlite3 (si se usa SQLite)

php8.2-mysql (si se usa MySQL/MariaDB)

php8.2-zip

php8.2-bcmath

php8.2-intl

2.3 Requisitos adicionales para servidor CS:GO local

Si el servidor de Counter-Strike se ejecuta en la misma máquina:

lib32gcc-s1

lib32stdc++6

Estas librerías de 32 bits son necesarias para el binario del servidor CS:GO Legacy.

3. Instalación en Linux

3.1 Instalar dependencias del sistema

```
sudo apt update && sudo apt upgrade -y
```

```
sudo apt install -y \
```

```
git curl unzip \
```

```
nginx \
```

```
sqlite3 \
```

```
php8.2 php8.2-cli php8.2-fpm \
```

```
php8.2-mbstring php8.2-xml php8.2-curl \
```

```
php8.2-sqlite3 php8.2-mysql php8.2-zip \
```

```
php8.2-bcmath php8.2-intl \  
composer \  
nodejs npm
```

Si se va a ejecutar el servidor CS:GO en la misma máquina:

```
sudo dpkg --add-architecture i386  
sudo apt update  
sudo apt install -y lib32gcc-s1 lib32stdc++6
```

3.2 Clonar el repositorio

```
cd /var/www  
sudo git clone <URL_DEL_REPOSITORIO> afterreload  
sudo chown -R www-data:www-data afterreload  
cd afterreload
```

3.3 Instalar dependencias PHP

```
composer install --no-dev --optimize-autoloader
```

> En entornos de desarrollo usa `composer install` sin flags.

3.4 Instalar dependencias Node.js

```
npm install
```

3.5 Crear y configurar el fichero de entorno

```
cp .env.example .env
```

```
php artisan key:generate
```

Edita `.env` con los valores de tu entorno. Ver sección [5. Configuración del entorno](#5-configuración-del-entorno-env) para referencia completa.

3.6 Preparar la base de datos

****Con SQLite (desarrollo / instalaciones pequeñas):****

```
touch database/database.sqlite
```

```
php artisan migrate --force
```

****Con MySQL (producción):****

Crear la base de datos en MySQL primero

```
mysql -u root -p -e "
```

```
CREATE DATABASE afterreload CHARACTER SET utf8mb4 COLLATE  
utf8mb4_unicode_ci;
```

```
CREATE USER 'afterreload'@'localhost' IDENTIFIED BY 'TU_PASSWORD';
```

```
GRANT ALL PRIVILEGES ON afterreload.* TO 'afterreload'@'localhost';
```

```
FLUSH PRIVILEGES;
```

```
"
```

Configurar en .env:

DB_CONNECTION=mysql

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=afterreload

DB_USERNAME=afterreload

DB_PASSWORD=TU_PASSWORD

`php artisan migrate --force`

3.7 Compilar el frontend

`npm run build`

Esto genera los assets compilados en `public/build/`.

3.8 Configurar permisos

`sudo chown -R www-data:www-data /var/www/afterreload/storage`

`sudo chown -R www-data:www-data /var/www/afterreload/bootstrap/cache`

`sudo chmod -R 775 /var/www/afterreload/storage`

`sudo chmod -R 775 /var/www/afterreload/bootstrap/cache`

3.9 Optimizar para producción

`php artisan config:cache`

`php artisan route:cache`

`php artisan view:cache`

3.10 Verificar la instalación

`php artisan about`

Este comando muestra el estado de la aplicación, versión de Laravel, configuración activa y extensiones cargadas.

4. Instalación en Windows

> Windows es válido para entornos de desarrollo local. Para producción se recomienda Linux.

4.1 Requisitos previos

Instala los siguientes programas:

- [Git for Windows](<https://git-scm.com/download/win>)
- [PHP 8.2+](<https://windows.php.net/download/>) — añadir al PATH del sistema
- [Composer](<https://getcomposer.org/Composer-Setup.exe>)
- [Node.js LTS](<https://nodejs.org/>) — incluye npm
- [SQLite](<https://www.sqlite.org/download.html>) (para desarrollo) o MySQL

4.2 Pasos de instalación

```
git clone <URL_DEL_REPOSITORIO> afterreload
```

```
cd afterreload
```

```
composer install
```

```
npm install
```

```
copy .env.example .env
```

```
php artisan key:generate
```

Si usas SQLite, crea el fichero:

```
type nul > database\database.sqlite
```

Luego:

```
php artisan migrate --force
```

```
npm run build
```

```
php artisan serve
```

La aplicación estará disponible en `http://localhost:8000`.

5. Configuración del entorno (.env)

El fichero `.env` controla toda la configuración del sistema. Nunca subas este fichero al repositorio Git (está excluido en `.gitignore`).

5.1 Configuración de la aplicación

APP_NAME=AfterReload

Nombre que aparece en el título del navegador y correos del sistema

APP_ENV=production

Valores: local | staging | production

En producción evita exponer información de depuración

APP_KEY=

Clave de cifrado de 32 bytes. Generada con: php artisan key:generate

CRÍTICO: sin esta clave las sesiones y datos cifrados no funcionan

APP_DEBUG=false

En producción debe ser false. Con true se muestran trazas de error al usuario

APP_URL=https://tu-dominio.com

URL pública completa de la aplicación, incluyendo protocolo

Usada en callbacks OAuth y generación de URLs internas

5.2 Configuración de logs

LOG_CHANNEL=stack

LOG_STACK=single

LOG_LEVEL=warning

En producción usar warning o error para reducir volumen de logs

En desarrollo usar debug para máximo detalle

Los logs se escriben en `storage/logs/laravel.log`.

5.3 Configuración de base de datos

DB_CONNECTION=sqlite

Valores: sqlite | mysql | mariadb | pgsql

Solo necesario si DB_CONNECTION=mysql o mariadb:

DB_HOST=127.0.0.1

DB_PORT=3306

DB_DATABASE=afterreload

DB_USERNAME=afterreload

DB_PASSWORD=tu_password_seguro

Ver sección [6. Configuración de la base de datos](#6-configuración-de-la-base-de-datos) para más detalles.

5.4 Configuración de sesiones, caché y cola

SESSION_DRIVER=database

Valores: file | cookie | database | redis | memcached

Con database las sesiones se guardan en la tabla sessions

SESSION_LIFETIME=120

Duración de la sesión en minutos

CACHE_STORE=database

Valores: file | database | redis | memcached

El sistema usa caché para locks de procesamiento de logs

QUEUE_CONNECTION=database

Valores: sync | database | redis

Para jobs en background (si se usan en el futuro)

5.5 Configuración de Steam OAuth

STEAM_CLIENT_ID=

No se usa actualmente (Steam no requiere client_id en su OAuth)

Dejar vacío

STEAM_CLIENT_SECRET=tu_api_key_de_steam

**Tu Steam Web API Key. Obtenida en:
<https://steamcommunity.com/dev/apikey>**

Necesaria para validar la autenticación de usuarios

STEAM_REDIRECT_URL=<https://tu-dominio.com/login/steam/callback>

URL de retorno tras autenticación. Debe coincidir exactamente con la configurada

en el portal de Steam. Cambia el dominio por el tuyo.

Ver sección [9. Integración con Steam OAuth](#9-integración-con-steam-oauth) para el proceso completo de obtención de credenciales.

5.6 Configuración de administradores

ADMIN_STEAM_IDS=76561198000000000,76561198111111111

Lista de SteamID64 separados por comas que tendrán rol de admin

automáticamente al hacer login, sin depender de la base de datos.

Útil para recuperar acceso de admin si se pierde en la BD.

ADMIN_STEAM_ID=76561198000000000

Alternativa para un solo admin (si no se usa ADMIN_STEAM_IDS)

Para encontrar tu SteamID64 visita: `https://steamid.io`` e introduce tu URL de perfil de Steam.

5.7 Configuración de Get5

GET5_WEBHOOK_TOKEN=token_secreto_aleatorio

Token de autenticación que Get5 enviará en la cabecera de sus peticiones.

Genera uno seguro con: `openssl rand -hex 32`

Debe configurarse también en el servidor CS:GO en el plugin Get5.

GET5_DEFAULT_MAP=de_mirage

Mapa por defecto para partidas matchmaking si no se especifica otro.

Ver sección [10. Integración con Get5](#10-integración-con-get5) para la configuración del plugin.

5.8 Configuración RCON

RCON_HOST=127.0.0.1

IP del servidor CS:GO al que se enviarán comandos RCON.

Si el servidor está en la misma máquina, usar 127.0.0.1

RCON_PORT=27015

Puerto RCON del servidor. Normalmente coincide con el puerto de juego.

RCON_PASSWORD=tu_rcon_password

Contraseña RCON configurada en el servidor CS:GO (rcon_password en server.cfg)

5.9 Configuración de la base de datos del servidor (opcional)

Solo necesaria si se quiere sincronizar rangos con SourceMod (plugin `lvl\_ranks`):

DB_SERVER_HOST=127.0.0.1

DB_SERVER_PORT=3306

DB_SERVER_DATABASE=afterreload_server

DB_SERVER_USERNAME=afterreload_user

DB_SERVER_PASSWORD=tu_password

5.10 Configuración de broadcasting (tiempo real)

Por defecto el broadcasting está desactivado (`log`). Para activar actualizaciones en tiempo real vía WebSockets:

BROADCAST_CONNECTION=pusher

PUSHER_APP_ID=tu_app_id

PUSHER_APP_KEY=tu_app_key

PUSHER_APP_SECRET=tu_app_secret

```
PUSHER_APP_CLUSTER=eu  
PUSHER_APP_HOST=  
PUSHER_APP_PORT=  
PUSHER_APP_SCHEME=https
```

6. Configuración de la base de datos

6.1 SQLite (desarrollo / pequeña escala)

SQLite no requiere servidor de base de datos. La base de datos es un único fichero en `database/database.sqlite`.

****Ventajas:**** Sin configuración de servidor, fácil de respaldar (copiar el fichero).

****Limitaciones:**** No apto para múltiples procesos concurrentes de escritura intensiva.

```
touch database/database.sqlite
```

```
php artisan migrate
```

6.2 MySQL / MariaDB (producción)

****Crear base de datos y usuario:****

```
CREATE DATABASE afterreload  
CHARACTER SET utf8mb4  
COLLATE utf8mb4_unicode_ci;
```



```
CREATE USER 'afterreload'@'localhost'  
IDENTIFIED BY 'contraseña_segura';  
  
GRANT ALL PRIVILEGES ON afterreload.*  
TO 'afterreload'@'localhost';  
  
FLUSH PRIVILEGES;
```

****Variables `.env` correspondientes:****

```
DB_CONNECTION=mysql  
DB_HOST=127.0.0.1  
DB_PORT=3306  
DB_DATABASE=afterreload  
DB_USERNAME=afterreload  
DB_PASSWORD=contraseña_segura
```

6.3 Esquema de la base de datos

Las migraciones se encuentran en `database/migrations/` y se ejecutan en orden cronológico. El estado actual incluye las siguientes tablas:

Tabla Descripción

#####

`users`	Jugadores registrados con datos Steam, rol y progresión
---------	---

`servers`	Servidores de Counter-Strike configurados
-----------	---

`lobbies`	Salas de espera asociadas a servidores
-----------	--

`lobby\_user`	Tabla pivote: relación N:M entre usuarios y lobbies
`lobby\_matches`	Configuraciones de partidas activas (Get5)
`match\_results`	Historial de resultados procesados
`sessions`	Sesiones activas de usuario
`cache`	Entradas de caché de la aplicación
`jobs`	Cola de trabajos en background
`password\_reset\_tokens`	Tokens de recuperación de contraseña

****Ejecutar migraciones:****

php artisan migrate

****Revertir la última migración:****

php artisan migrate:rollback

****Resetear toda la base de datos (A destructivo):****

php artisan migrate:fresh

7. Configuración de Nginx

7.1 Crear el fichero de configuración del sitio

sudo nano /etc/nginx/sites-available/afterreload

7.2 Configuración básica (HTTP)

```
server {  
  
    listen 80;  
  
    server_name tu-dominio.com www.tu-dominio.com;  
  
    root /var/www/afterreload/public;  
    index index.php;  
  
    charset utf-8;  
  
    location / {  
        try_files $uri $uri/ /index.php?$query_string;  
    }  
  
    location = /favicon.ico { access_log off; log_not_found off; }  
    location = /robots.txt  { access_log off; log_not_found off; }  
  
    error_page 404 /index.php;  
  
    location ~ \.php$ {  
        fastcgi_pass unix:/run/php/php8.2-fpm.sock;  
        fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;  
        include fastcgi_params;  
        fastcgi_hide_header X-Powered-By;  
    }  
}
```

```
location ~ /\.(!well-known).* {  
    deny all;  
}  
  
# Caché de assets compilados por Vite  
location /build/ {  
    expires 1y;  
    add_header Cache-Control "public, immutable";  
}  
}
```

7.3 Activar el sitio

```
sudo ln -s /etc/nginx/sites-available/afterreload /etc/nginx/sites-enabled/  
sudo nginx -t  
sudo systemctl reload nginx
```

7.4 Configuración con HTTPS (Certbot)

```
sudo apt install -y certbot python3-certbot-nginx  
sudo certbot --nginx -d tu-dominio.com -d www.tu-dominio.com
```

Certbot modifica automáticamente la configuración de Nginx para añadir el bloque SSL y la redirección HTTP → HTTPS.

8. Configuración de PHP-FPM

8.1 Fichero de pool por defecto

```
sudo nano /etc/php/8.2/fpm/pool.d/www.conf
```

Ajustes recomendados para un servidor con 1–2 GB de RAM:

```
[www]

user = www-data

group = www-data

listen = /run/php/php8.2-fpm.sock

listen.owner = www-data

listen.group = www-data


pm = dynamic

pm.max_children = 10

pm.start_servers = 3

pm.min_spare_servers = 2

pm.max_spare_servers = 5

pm.max_requests = 500
```

8.2 Configuración de PHP

Fichero: `/etc/php/8.2/fpm/php.ini`

```
upload_max_filesize = 10M
```

```
post_max_size = 10M  
max_execution_time = 30  
memory_limit = 256M  
date.timezone = Europe/Madrid
```

8.3 Reiniciar PHP-FPM

```
sudo systemctl restart php8.2-fpm  
sudo systemctl enable php8.2-fpm
```

9. Integración con Steam OAuth

AfterReload utiliza Steam como único proveedor de autenticación. El paquete `socialiteproviders/steam` gestiona el flujo OAuth.

9.1 Obtener Steam Web API Key

20. 1. Inicia sesión en Steam en tu navegador.
21. 2. Accede a: `https://steamcommunity.com/dev/apikey`
22. 3. Introduce el dominio de tu aplicación en el campo "Domain Name" (por ejemplo: `tu-dominio.com`).
23. 4. Acepta los términos y haz clic en "Register".
24. 5. Copia la ****API Key**** generada.

9.2 Configurar las variables de entorno

```
STEAM_CLIENT_SECRET=TU_STEAM_API_KEY  
STEAM_REDIRECT_URL=https://tu-dominio.com/login/steam/callback
```

9.3 Flujo OAuth implementado

Usuario → GET /login/steam

- Redirige a <https://steamcommunity.com/openid/login>
- Usuario se autentica en Steam
- Steam redirige a GET /login/steam/callback
- SteamController::handleSteamCallback()
- User::updateOrCreate(['steam_id' => ...], [...])
- Auth::login(\$user)
- Redirect a /servers

9.4 Gestión de administradores por SteamID

Los SteamIDs en `ADMIN\_STEAM\_IDS` se verifican en cada petición por el método `User::isAdmin()`. Esto permite que un usuario tenga privilegios de administrador incluso si su rol en base de datos es `user`, actuando como fallback de seguridad para recuperar el acceso administrativo.

10. Integración con Get5

Get5 es un plugin de SourceMod para Counter-Strike que gestiona partidas competitivas. AfterReload actúa como servidor de configuración y receptor de eventos de Get5.

10.1 Requisitos en el servidor CS:GO

El servidor de Counter-Strike debe tener instalado:

- ****SourceMod**** (1.11 o superior)
- ****MetaMod: Source****
- ****Get5**** (versión compatible con CS:GO)

10.2 Configuración de Get5 en el servidor

En el fichero de configuración de Get5 (`addons/sourcemod/configs/get5/`), asegúrate de que los webhooks están activos. En el `server.cfg` o configuración de Get5:

```
// URL del webhook de eventos
```

```
get5_remote_log_url "https://tu-dominio.com/api/get5/events"
```

```
// Token de autenticación (debe coincidir con GET5_WEBHOOK_TOKEN en .env)
```

```
get5_remote_log_header_value "Bearer TU_GET5_WEBHOOK_TOKEN"
```

```
get5_remote_log_header_key "Authorization"
```

10.3 Endpoint de configuración de match

Cuando Get5 necesita cargar la configuración de una partida, consulta:

```
GET /api/get5/match/{lobby_id}
```

Devuelve un JSON con el formato que Get5 espera:

```
{
  "matchid": "match-uuid",
  "num_maps": 1,
```



```
"maplist": ["de_mirage"],
"team1": {
  "name": "Counter-Terrorists",
  "players": {
    "76561198000000001": "Jugador1",
    "76561198000000002": "Jugador2"
  }
},
"team2": {
  "name": "Terrorists",
  "players": {
    "76561198000000003": "Jugador3",
    "76561198000000004": "Jugador4"
  }
},
"cvvars": {
  "get5_remote_log_url": "https://tu-dominio.com/api/get5/events",
  "get5_remote_log_header_key": "Authorization",
  "get5_remote_log_header_value": "Bearer TU_TOKEN"
}
}
```

10.4 Endpoint de eventos (webhook)

POST /api/get5/events

Authorization: Bearer TU_GET5_WEBHOOK_TOKEN

Content-Type: application/json

AfterReload procesa los siguientes tipos de eventos:

| Evento | Acción |

#####

| `series\_result` | Aplica puntos a ganadores/perdedores y cierra el lobby |

| `map\_result` | Registra el resultado del mapa |

| `player\_death` | (registrado pero no procesado actualmente) |

| Otros | Almacenados en `match\_results` para auditoría |

10.5 Deduplicación de eventos

Cada evento recibido genera un hash SHA-256 del payload completo. Antes de procesarlo, el sistema verifica que ese hash no existe ya en la tabla `match\_results`. Si existe, el evento se descarta silenciosamente. Esto previene la doble aplicación de puntos si Get5 reenvía el mismo evento.

10.6 Aplicación de puntos (matchmaking)

Al recibir un `series\_result`:

- **Equipo ganador**: +X puntos por jugador (configurable en `Get5Controller::applyPoints()`)
- **Equipo perdedor**: -Y puntos por jugador

Los puntos y créditos se actualizan directamente en la tabla `users`.

11. Integración RCON

El cliente RCON permite enviar comandos de consola al servidor CS:GO de forma remota desde la aplicación.

11.1 Configurar RCON en el servidor CS:GO

En el fichero `server.cfg` del servidor de Counter-Strike:

```
rcon_password "tu_rcon_password"
```

****Importante:**** Usa una contraseña larga y aleatoria. RCON da control total sobre el servidor.

11.2 Variables de entorno RCON

```
RCON_HOST=127.0.0.1 # IP del servidor CS:GO
```

```
RCON_PORT=27015 # Puerto del servidor (no del RCON, es el mismo)
```

```
RCON_PASSWORD=tu_rcon_password
```

11.3 Funcionamiento del cliente

`RconClient` implementa el protocolo RCON de Valve sobre TCP. La autenticación y el envío de comandos se realizan en el mismo socket:

25. 1. Abre conexión TCP al servidor.
26. 2. Envía paquete de autenticación (tipo 3) con la contraseña.
27. 3. Verifica que el servidor responde con el mismo ID de paquete.
28. 4. Envía el comando (tipo 2).
29. 5. Lee la respuesta del servidor.
30. 6. Cierra el socket.

El timeout por defecto es de **1.5 segundos**. Si el servidor no responde en ese tiempo, `send()` devuelve `null` sin lanzar excepción.

11.4 Comandos RCON utilizados por AfterReload

| Momento | Comando enviado |

#####

| Inicio de match | `get5 load_match_url https://tu-dominio.com/api/get5/match/{id}` |

| Fin de match | Comandos de reset de configuración del servidor |

12. Integración con servidor CS:GO local (LGSM)

Si el servidor CS:GO se ejecuta en la misma máquina usando **LinuxGSM (LGSM)**, AfterReload puede detectar automáticamente si el servidor está activo mediante los ficheros de lock que LGSM crea cuando el servidor arranca.

12.1 Configuración de la ruta de locks

`CSGOSERVER_LOCK_DIR=/var/www/afterreload/csgoserver/lgsm/lock`

Ruta al directorio donde LGSM crea sus ficheros .lock

En `config/services.php`:

```
'csgoserver' => [  
    'lock_dir' => env('CSGOSERVER_LOCK_DIR', '/var/www/afterreload/csgoserver/lgsm/lock'),  
    'port_map' => [  

```

```

    27015 => 'csgoserver-started.lock',

    27016 => 'csgoserver-wingman-started.lock',

    27017 => 'mmserver-started.lock',

],
],

```

12.2 Mapeo de puertos a ficheros de lock

Cada puerto de servidor se mapea a un fichero de lock específico. Cuando LGSM inicia una instancia de servidor, crea el fichero de lock correspondiente. Cuando la detiene, lo elimina.

Puerto	Fichero de lock	Tipo de servidor
27015	csgoserver-started.lock	Público 5v5
27016	csgoserver-wingman-started.lock	Wingman 2v2
27017	mmserver-started.lock	Matchmaking competitivo

12.3 Añadir más servidores

Para añadir un nuevo servidor al mapeo, edita `config/services.php`:

```

'port_map' => [

    27015 => 'csgoserver-started.lock',

    27016 => 'csgoserver-wingman-started.lock',

    27017 => 'mmserver-started.lock',

    27018 => 'csgoserver-nuevo-started.lock', // nuevo servidor

],

```

12.4 Procesamiento de logs públicos

`PublicMatchResultProcessor` escanea los logs del servidor CS:GO para detectar el final de partidas en servidores públicos. Los logs se buscan en la carpeta de logs del servidor de juego.

El servicio determina el umbral de rondas para ganar según el puerto del servidor:

- Puerto **27016** (Wingman): umbral de **9 rondas**.
- Cualquier otro puerto: umbral de **16 rondas** (5v5 estándar).

13. Compilación del frontend

El frontend usa **Vite 7** con **Tailwind CSS 4** para compilar los assets.

13.1 Compilar para producción

```
npm run build
```

Genera los ficheros en `public/build/`. El `manifest.json` generado en ese directorio es leído por el helper `@vite()` de Blade para enlazar los assets con hash de contenido (cache-busting).

13.2 Servidor de desarrollo

```
npm run dev
```

Inicia el servidor de desarrollo de Vite con Hot Module Replacement (HMR). Requiere que la aplicación Laravel también esté corriendo. Para arrancar todo el stack de desarrollo:

composer run dev

Este comando ejecuta en paralelo: servidor PHP, worker de cola, visor de logs (Pail) y Vite.

13.3 Estructura de assets

resources/

```
|— css/
|   └─ app.css      # Entrada de Tailwind CSS
└─ js/
    └─ app.js        # Entrada JavaScript principal
```

public/

```
└─ build/
    └─ manifest.json # Mapa de assets con hashes
    └─ assets/
        └─ app-[hash].css
        └─ app-[hash].js
    └─ ...
```

14. Comandos de mantenimiento

14.1 Artisan — referencia de comandos útiles

Estado general de la aplicación

`php artisan about`

Ejecutar migraciones pendientes

`php artisan migrate`

Revertir última migración

`php artisan migrate:rollback`

Recrear toda la base de datos (borra todos los datos)

`php artisan migrate:fresh`

Limpiar caché de configuración

`php artisan config:clear`

Limpiar caché de rutas

`php artisan route:clear`

Limpiar caché de vistas compiladas

`php artisan view:clear`

Limpiar toda la caché de la aplicación

`php artisan cache:clear`

Regenerar caché de configuración (producción)

`php artisan config:cache`

Regenerar caché de rutas (producción)

`php artisan route:cache`

Regenerar caché de vistas (producción)

`php artisan view:cache`

Optimizar todo para producción (equivale a los tres anteriores)

`php artisan optimize`

Revertir optimizaciones

`php artisan optimize:clear`

Ejecutar todos los tests

`php artisan test`

Ejecutar tests en paralelo

`php artisan test --parallel`

Ver logs en tiempo real (requiere Laravel Pail)

`php artisan pail`

Acceder a la consola interactiva (Tinker)

php artisan tinker

Activar modo de mantenimiento

php artisan down

Desactivar modo de mantenimiento

php artisan up

14.2 Scripts de Composer

Instalación completa desde cero

composer run setup

Entorno de desarrollo completo (server + queue + logs + vite)

composer run dev

Ejecutar tests

composer run test

15. Gestión de logs

15.1 Ubicación de logs

storage/

└─ logs/

└─ laravel.log # Log principal de la aplicación

15.2 Ver logs en tiempo real

Con Laravel Pail (recomendado en desarrollo)

```
php artisan pail
```

Con tail estándar

```
tail -f storage/logs/laravel.log
```

Solo errores

```
tail -f storage/logs/laravel.log | grep "ERROR"
```

15.3 Nivel de log en producción

En producción, establece el nivel mínimo en ``.env``:

```
LOG_LEVEL=warning
```

Esto registra únicamente mensajes de nivel ``warning``, ``error`` y ``critical``, reduciendo el volumen de logs.

15.4 Rotación de logs

Para evitar que el fichero de log crezca indefinidamente, cambia el canal a ``daily`` en ``.env``:

LOG_CHANNEL=daily

Con `daily`, Laravel crea un nuevo fichero de log cada día con el formato `laravel-YYYY-MM-DD.log` y mantiene los últimos 14 días por defecto.

15.5 Limpiar logs

Vaciar el log actual (sin eliminar el fichero)

```
> storage/logs/laravel.log
```

Eliminar logs antiguos (modo daily)

```
find storage/logs/ -name "laravel-*.log" -mtime +30 -delete
```

16. Gestión de la cola de trabajos

AfterReload usa el driver de cola `database` por defecto. Los jobs pendientes se almacenan en la tabla `jobs`.

16.1 Ejecutar el worker de cola

```
php artisan queue:work
```

Para producción, usa un gestor de procesos como **Supervisor**:

```
sudo apt install -y supervisor
```

```
sudo nano /etc/supervisor/conf.d/afterreload-queue.conf
```

```
[program:afterreload-queue]
```

```
process_name=%(program_name)s_%(process_num)02d
```

```
command=php /var/www/afterreload/artisan queue:work --sleep=3 --tries=3 --max-time=3600
```

```
autostart=true
```

```
autorestart=true
```

```
stopasgroup=true
```

```
killasgroup=true
```

```
user=www-data
```

```
numprocs=1
```

```
redirect_stderr=true
```

```
stdout_logfile=/var/www/afterreload/storage/logs/queue.log
```

```
stopwaitsecs=3600
```

```
sudo supervisorctl reread
```

```
sudo supervisorctl update
```

```
sudo supervisorctl start afterreload-queue:*
```

16.2 Monitorizar la cola

Ver jobs fallidos

```
php artisan queue:failed
```

Reintentar todos los jobs fallidos

php artisan queue:retry all

Vaciar jobs fallidos

php artisan queue:flush

17. Seguridad

17.1 Variables de entorno sensibles

Nunca expongas en el repositorio ni en logs los siguientes valores:

- `APP\_KEY` — Clave maestra de cifrado de Laravel.
- `STEAM\_CLIENT\_SECRET` — API Key de Steam.
- `GET5\_WEBHOOK\_TOKEN` — Token de autenticación del webhook.
- `RCON\_PASSWORD` — Contraseña de control remoto del servidor.
- `DB\_PASSWORD` — Contraseña de la base de datos.

17.2 Cabeceras de seguridad HTTP

Añade las siguientes cabeceras en la configuración de Nginx:

```
add_header X-Frame-Options "SAMEORIGIN";
```

```
add_header X-Content-Type-Options "nosniff";
```

```
add_header X-XSS-Protection "1; mode=block";
```

```
add_header Referrer-Policy "strict-origin-when-cross-origin";
```

17.3 Protección de ficheros sensibles

El bloque de Nginx `location ~ /\.`` ya impide el acceso a ficheros ocultos como ``.env``. Verifica que esta regla esté activa:

```
location ~ /\.(!well-known).* {  
    deny all;  
}
```

17.4 Permisos del sistema de ficheros

Solo `storage/`` y `bootstrap/cache/`` deben ser escribibles por el proceso web:

```
sudo chmod -R 755 /var/www/afterreload  
sudo chmod -R 775 /var/www/afterreload/storage  
sudo chmod -R 775 /var/www/afterreload/bootstrap/cache  
sudo chown -R www-data:www-data /var/www/afterreload/storage  
sudo chown -R www-data:www-data /var/www/afterreload/bootstrap/cache
```

17.5 RCON — consideraciones de seguridad

- La comunicación RCON no está cifrada. En un entorno donde el servidor de juego está en una máquina diferente, asegúrate de que el puerto RCON no sea accesible desde internet (usa un firewall o túnel SSH).
- Si el servidor CS:GO está en la misma máquina que la web, usa ``RCON_HOST=127.0.0.1`` para limitar la comunicación a localhost.

17.6 Webhook de Get5 — autenticación

El endpoint `POST /api/get5/events` verifica el token `Authorization: Bearer TOKEN` en cada petición. Asegúrate de que `GET5_WEBHOOK_TOKEN` sea un valor aleatorio y largo (mínimo 32 caracteres):

```
openssl rand -hex 32
```

18. Resolución de problemas

18.1 La aplicación muestra un error 500

31. 1. Comprueba que `APP_KEY` está configurado en `.env`.
32. 2. Revisa los logs: `tail -f storage/logs/laravel.log`.
33. 3. Asegúrate de que los permisos de `storage/` son correctos.
34. 4. Verifica que las extensiones PHP requeridas están instaladas: `php -m`.

18.2 El login con Steam falla

35. 1. Verifica que `STEAM_CLIENT_SECRET` es una Steam Web API Key válida.
36. 2. Comprueba que `STEAM_REDIRECT_URL` coincide exactamente con la URL pública de tu aplicación (incluyendo el protocolo `https://`).
37. 3. Asegúrate de que `APP_URL` en `.env` está correctamente configurado.
38. 4. Verifica que el dominio registrado en Steam API coincide con el que está sirviendo la aplicación.

18.3 Las sesiones no persisten entre peticiones

39. 1. Comprueba que `SESSION_DRIVER=database` está configurado en `.env`.
40. 2. Verifica que la tabla `sessions` existe en la base de datos (`php artisan migrate`).
41. 3. Asegúrate de que el driver de sesión tiene permisos de escritura (si usas `file`, verifica permisos de `storage/framework/sessions/`).

18.4 Get5 no envía eventos al webhook

42. 1. Verifica que la URL del webhook en la configuración de Get5 es accesible desde el servidor de CS:GO.
43. 2. Comprueba que el token en `get5\_remote\_log\_header\_value` coincide con `GET5\_WEBHOOK\_TOKEN` en `.env`.
44. 3. Revisa los logs de Laravel para ver si las peticiones llegan: `tail -f storage/logs/laravel.log`.
45. 4. Prueba el endpoint manualmente con curl:

```
curl -X POST https://tu-dominio.com/api/get5/events \  
  
-H "Authorization: Bearer TU_TOKEN" \  
  
-H "Content-Type: application/json" \  
  
-d '{"event":"test"}'
```

18.5 RCON no responde

46. 1. Verifica que el servidor CS:GO está en ejecución.
47. 2. Comprueba que `rcon\_password` en `server.cfg` del servidor coincide con `RCON\_PASSWORD` en `.env`.
48. 3. Verifica que el puerto `RCON\_PORT` es accesible desde la máquina de la aplicación.
49. 4. Prueba la conectividad desde la consola de Laravel Tinker:

```
php artisan tinker
```

```
>>> app(App\Services\RconClient::class)->send('127.0.0.1', 27015, 'tu_password', 'status')
```

Si devuelve `null`, la conexión está fallando.

18.6 Los jugadores no se eliminan del lobby al cerrar el navegador

El sistema de poda funciona por TTL: elimina jugadores cuyo registro en `lobby\_user` no se ha actualizado en los últimos 20 segundos. Este proceso ocurre cuando otro usuario hace una petición al endpoint `/lobby/{server}/status`.

Si el lobby queda completamente vacío y nadie hace peticiones, los registros obsoletos persisten hasta que el próximo usuario acceda al lobby. Los administradores pueden usar la función "Limpiar lobbies" en el panel de admin para forzar la limpieza.

18.7 Error al compilar el frontend

Limpiar caché de Node

```
rm -rf node_modules/.cache
```

```
rm -rf public/build
```

Reinstalar dependencias

```
npm install
```

```
npm run build
```

18.8 La base de datos no se puede escribir (SQLite)

Verificar que el fichero existe

```
ls -la database/database.sqlite
```

Crear si no existe

```
touch database/database.sqlite
```

Corregir propietario

```
sudo chown www-data:www-data database/database.sqlite
```

```
sudo chmod 664 database/database.sqlite
```

19. Referencia de rutas del sistema

19.1 Rutas web

Método	Ruta	Controlador	Middleware	Descripción
GET	/	Closure	—	Página de bienvenida
GET	/lang/{locale}	LanguageController@switch	—	Cambiar idioma
GET	/login	Closure	—	Redirección a Steam
GET	/login/steam	SteamController@redirectToSteam	—	Iniciar OAuth Steam
GET	/login/steam/callback	SteamController@handleSteamCallback	—	Callback OAuth Steam
GET	/ranking	RankingController@index	—	Ranking público
GET	/contact	Closure	—	Página de contacto
GET	/servers	ServerController@index	auth	Lista de servidores
GET	/servers/data	ServerController@data	auth	JSON de servidores (polling)
GET	/lobby/{server}	LobbyController@show	auth	Vista de lobby
GET	/lobby/{server}/status	LobbyController@status	auth	JSON de estado del lobby
POST	/lobby/{server}/leave	LobbyController@leave	auth	Abandonar lobby
POST	/lobby/{server}/team	LobbyController@setTeam	auth	Seleccionar equipo
POST	/lobby/{server}/ready	LobbyController@toggleReady	auth	Alternar estado listo
GET	/admin	AdminController@index	auth, admin	Panel de administración
POST	/admin/servers	AdminController@storeServer	auth, admin	Crear servidor
PUT	/admin/servers/{server}	AdminController@updateServer	auth, admin	Editar servidor
DELETE	/admin/servers/{server}	AdminController@deleteServer	auth, admin	Eliminar servidor
POST	/admin/servers/clear-lobbies	AdminController@clearLobbies	auth, admin	Limpiar lobbies

POST	`/admin/users/{user}/role`	`AdminController@updateRole`	auth, admin	Cambiar rol
POST	`/admin/users/{user}/stats`	`AdminController@updateStats`	auth, admin	Editar puntos/créditos
POST	`/admin/users/{user}/ban`	`AdminController@toggleBan`	auth, admin	Banear usuario
POST	`/admin/users/{user}/unban`	`AdminController@unban`	auth, admin	Desbanear usuario
POST	`/logout`	`SteamController@logout`	auth	Cerrar sesión

19.2 Rutas API

Método	Ruta	Controlador	Autenticación	Descripción
---	---	---	---	---
POST	`/api/get5/events`	`Get5Controller@events`	Token Bearer	Webhook de eventos Get5
GET	`/api/get5/match/{lobby}`	`Get5Controller@match`	—	Configuración de match para Get5

19.3 Middleware disponibles

Middleware	Clase	Función
---	---	---
`auth`	Framework Laravel	Requiere sesión iniciada
`not_banned`	`EnsureNotBanned`	Redirige usuarios baneados a `/banned`
`set_locale`	`SetLocale`	Aplica idioma desde sesión
`admin`	(verificación en controlador)	Solo usuarios con rol admin o en ADMIN_STEAM_IDS